

# N-Queens Benchmark Suite

Elliot Klauke

*Keywords:* C, Constraint Satisfaction Problems (CSP), forward checking, AC-3, Minimum Remaining Values (MRV), Least Constraining Variable (LCV).

## 1 Purpose

The purpose of this project is to service classroom lessons on constraint satisfaction problems and algorithmic techniques such as filtering and ordering. Simulation data on the time and space efficiency of different combinations of algorithmic techniques is gathered and evaluated to conclude the benefits and takeaways of each method, as well as different specific cases that each might be used in. This data is formatted in different visualizations, including data tables, bar graphs, line graphs, and simulation replays, to demonstrate their performance to a student audience that may or may not be familiar with constraint satisfaction problems, certain algorithms, or algorithmic performance.

This project uses an example of the “N-Queens” problem, which describes a way to find placements of Queen chess pieces on a board size of  $n$  rows and columns such that there are  $n$  Queens placed that are not contesting each other.

## 2 Data and Methods

The solver represents the board as a vector of Queen objects associated with each row of the board. Each Queen object contains a vector of  $N$  boolean values representing the Queen’s domain of possible columns to be placed. I implemented and benchmarked the following algorithmic layers:

1. Backtracking:  
A recursive, depth-first algorithm that places new Queens or returns if a board is invalid. Backtracking is the framework of every algorithm combination tested.
2. Look-Ahead / Filtering Techniques:  
Forward Checking (FC): Prunes domains of unassigned variables ahead of time whenever a queen is placed.  
Arc Consistency (AC-3): Enforces directional arc consistency across all variable pairs.
3. Search Heuristics:  
Minimum Remaining Values (MRV): Variable ordering heuristic that selects the row with the fewest legal remaining columns (fail-first principle).  
Least Constraining Value (LCV): Value ordering heuristic that selects the column maximizing the total remaining unassigned variables across all Queens.

Each combination of filtering techniques and heuristics were run on board sizes 1-100. Combinations include:

| No Filtering/Heuristic | LCV       | MRV       | LCV + MRV       |
|------------------------|-----------|-----------|-----------------|
| FC                     | FC + LCV  | FC + MRV  | FC + LCV + MRV  |
| ARC                    | ARC + LCV | ARC + MRV | ARC + LCV + MRV |

Simulations that either returned "No Solution" or exceeded 10 seconds in real-time were not included in the results. Figures below contain the moving average for time (ms) and space (nodes expanded) against board size ( $n \times n$ ) on a logarithmic scale:

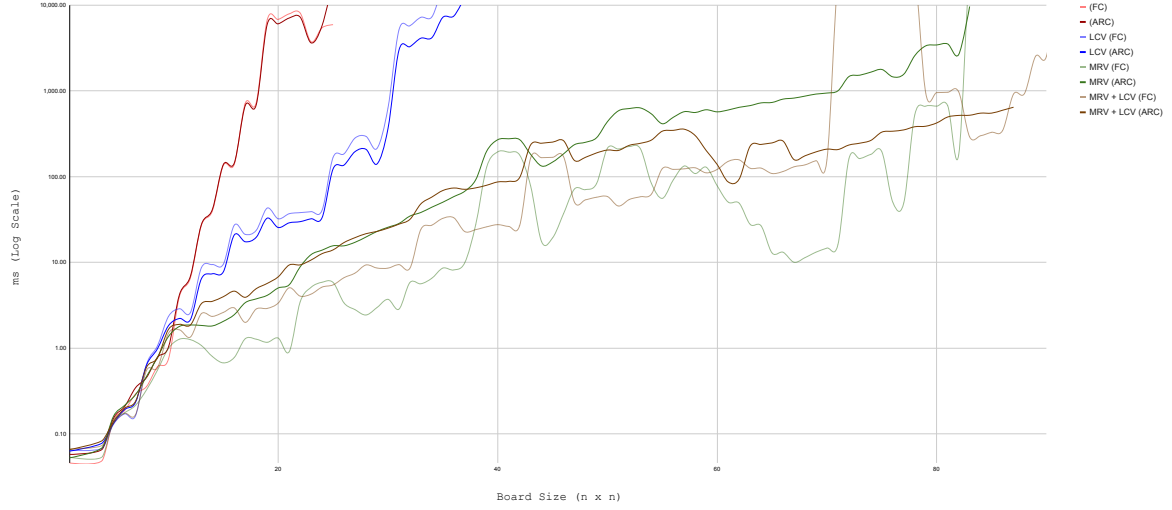


Figure 1: Time taken (ms) against board size.

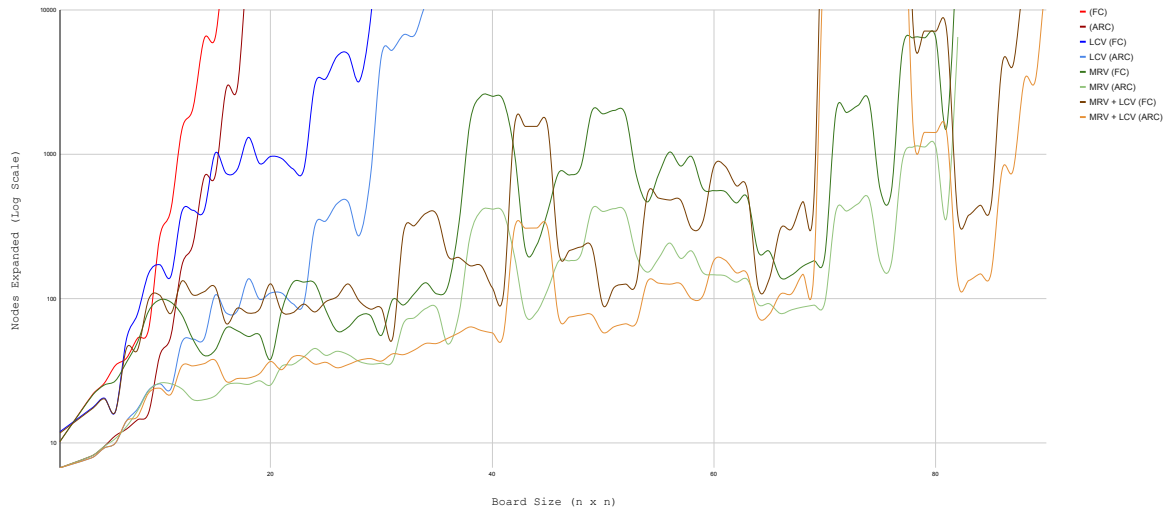


Figure 2: Nodes expanded against board size.

### 3 Individual Contribution, Principal Tools, & Languages Used

Each combination of algorithmic techniques was written and simulated by me, and the visualizations, explanations, and conclusions were created by me, with an exception of the Hennessy & Patterson [1] visual on processor and memory performance.

All algorithms were written in C and simulated on a Linux virtual machine.

### 4 Testing, Material Limitations, & Dependencies

Results of each algorithm were verified using a secondary program that checked the output against the original problem's constraints.

The results and conclusions about the algorithmic techniques used are strictly limited to the example of the N-Queens constraint satisfaction problem, and may or may not generalize to other constraint satisfaction problems.

#### **Availability of data and software code**

Software code: [https://github.com/eklauke/nqueens\\_suite](https://github.com/eklauke/nqueens_suite).

Presentation: [https://github.com/eklauke/nqueens\\_suite/blob/main/CSP%20NQueens.pptx](https://github.com/eklauke/nqueens_suite/blob/main/CSP%20NQueens.pptx)

#### **References**

- [1] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach (4th Edition)*. Morgan Kaufmann, 2006.